

UNIX V6 ++ 离散化



设计目标：将UNIX V6++的内存管理修改为离散化管理方式，不使用盘交换区

(1) 修改UNIX V6++的页表系统

(2) 将内存分配方式改为位示图

(3) 离散化存储



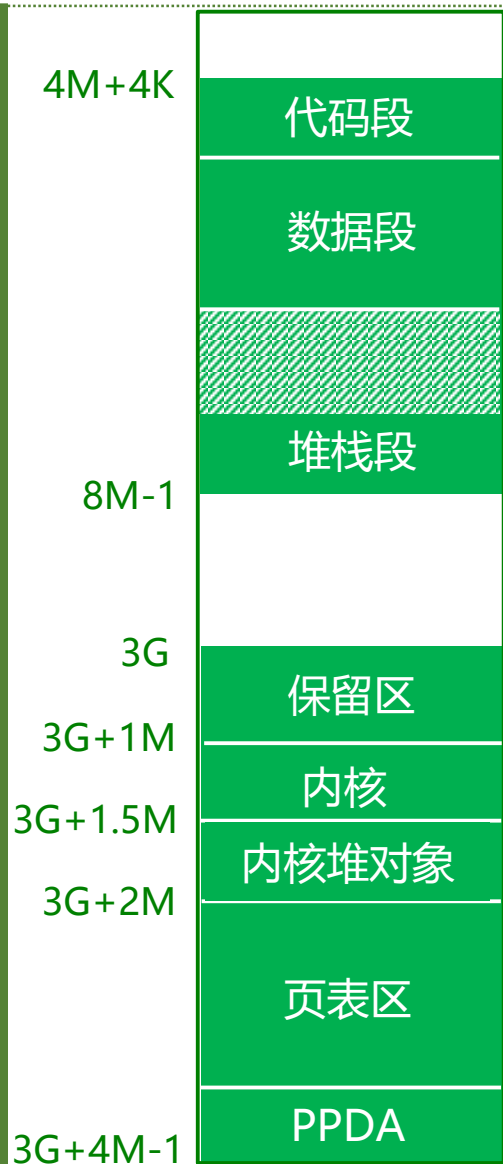
页表系统修改



页表系统



页表的构成



Page Table 0#

	Page Base Address		s/u	r/w	p
0#	/	/	/	/	/
...	...	...	...	...	...
1023#	/	/	/	/	/

Page Table 1#

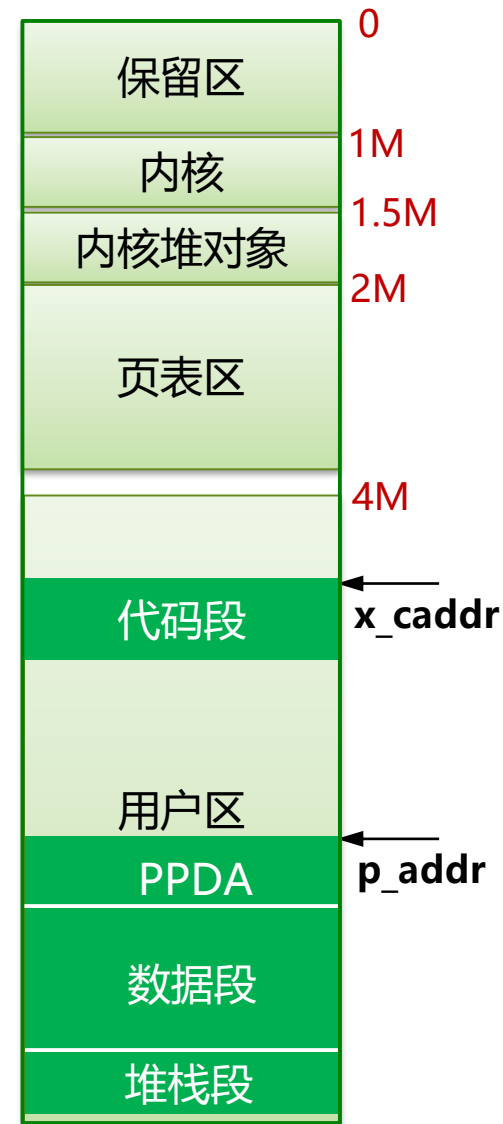
	Page Base Address		s/u	r/w	p
0#	/	/	/	/	/
1#	$x_caddr >> 12$	1	0	1	1
	$x_caddr >> 12+1$	1	0	1	1
.....	...	...	...	...	...
	$x_caddr >> 12+(n-1)$	1	0	1	1
	$p_addr >> 12+1$	1	1	1	1
.....	...	...	...	...	...
	$p_addr >> 12+1+(m-1)$	1	1	1	1
.....	...	...	...	...	...
1023#	$p_addr >> 12+1+m$	...	1	1	1

Page Table 768#

	Page Base Address		s/u	r/w	p
0#	0	...	0	1	1
1#	1	...	0	1	1
	2	...	0	1	1
.....	...	...	...	...	...
1022#	1022	...	0	1	1
1023#	$p_addr >> 12$	...	0	1	1

三张页表保存在哪里？

页目录保存在哪里？

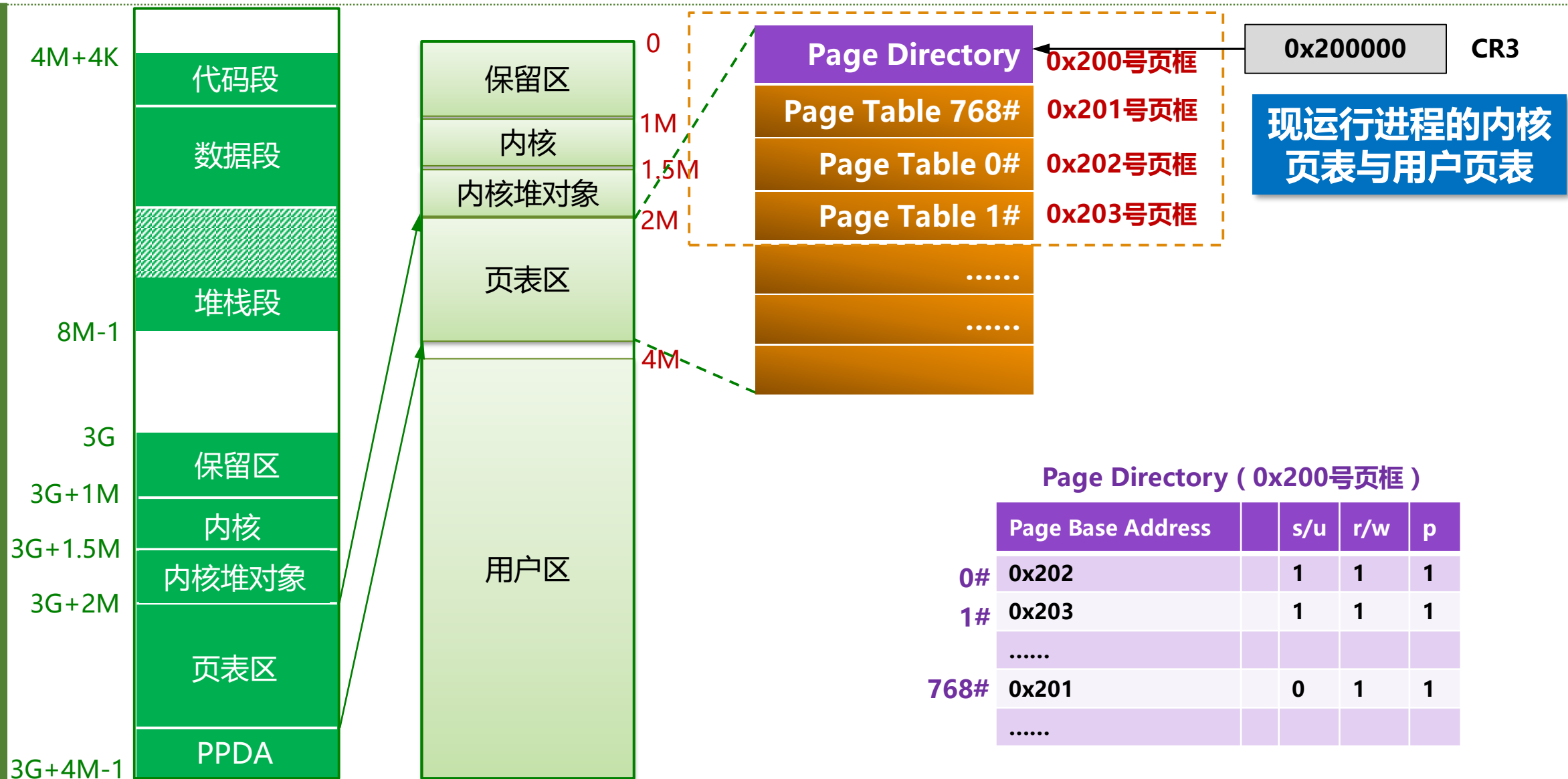




页表系统



页表的构成

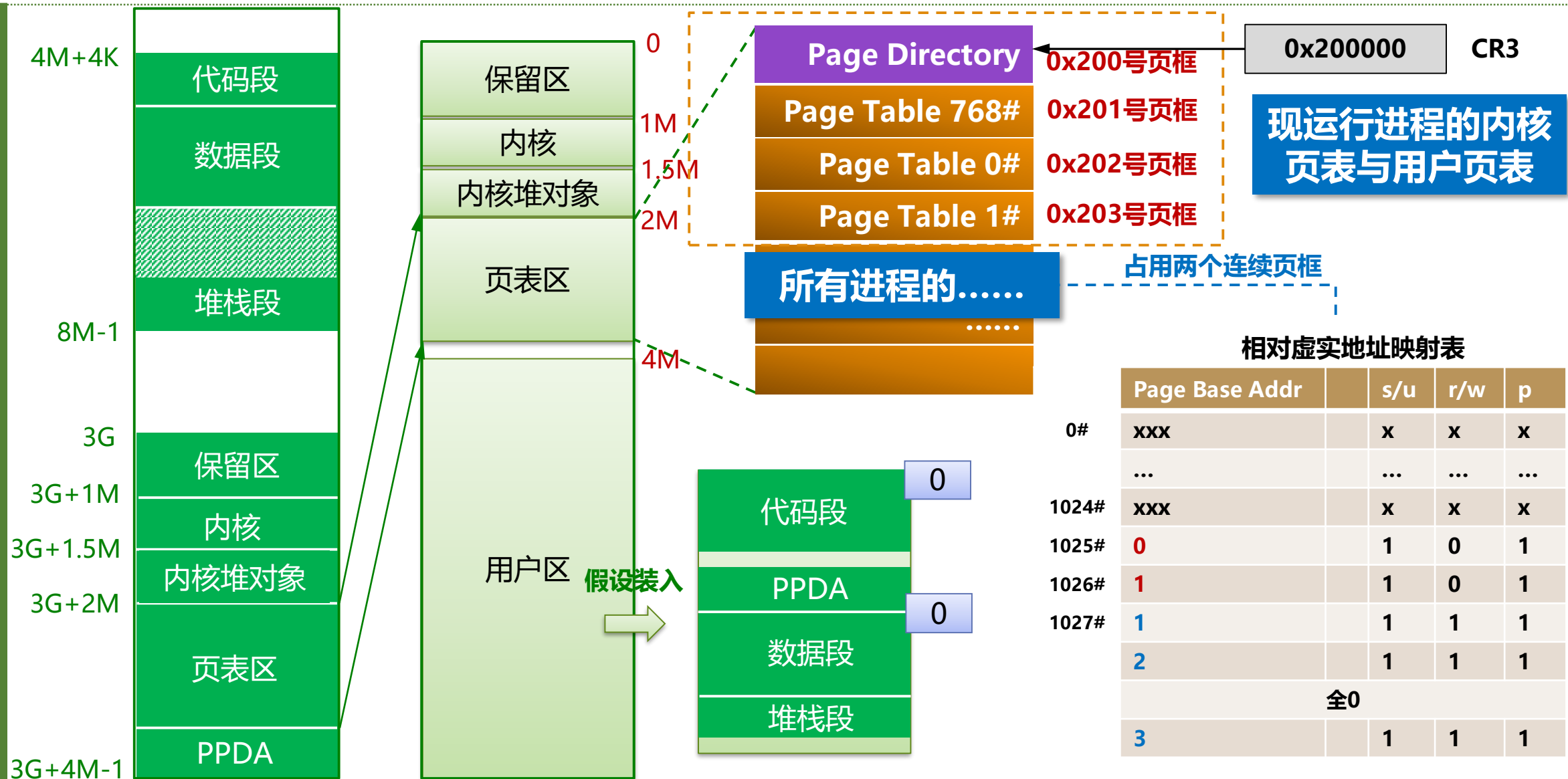




页表系统



页表的构成

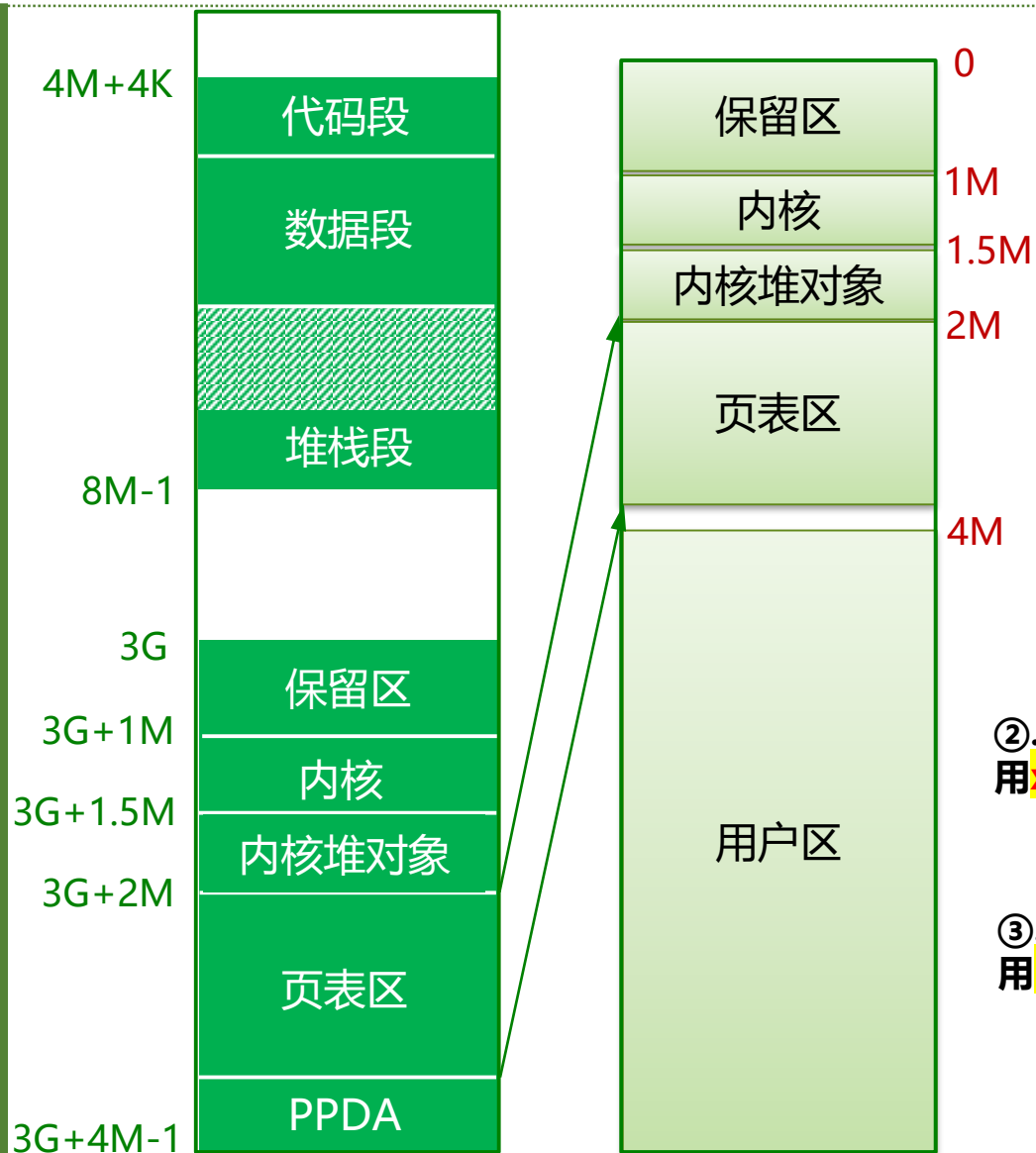




页表系统



页表的构成



虚实地址空间的分布沿用UNIX V6++的设计

修改要求：

(1) 不使用交换区；

(2) 直接使用相对表作为进程的物理页表

相对虚实地址映射表

	Page Base Addr	s/u	r/w	p
0#	xxx	x	x	x
...	...	...	...	...
1024#	xxx	x	x	x
1025#	0	1	0	1
1026#	1	1	0	1
1027#	1	1	1	1
	2	1	1	1
	全0			
2023#	3	1	1	1

②. if $r/w == 0$, 直接用 x_caddr 计算并写入

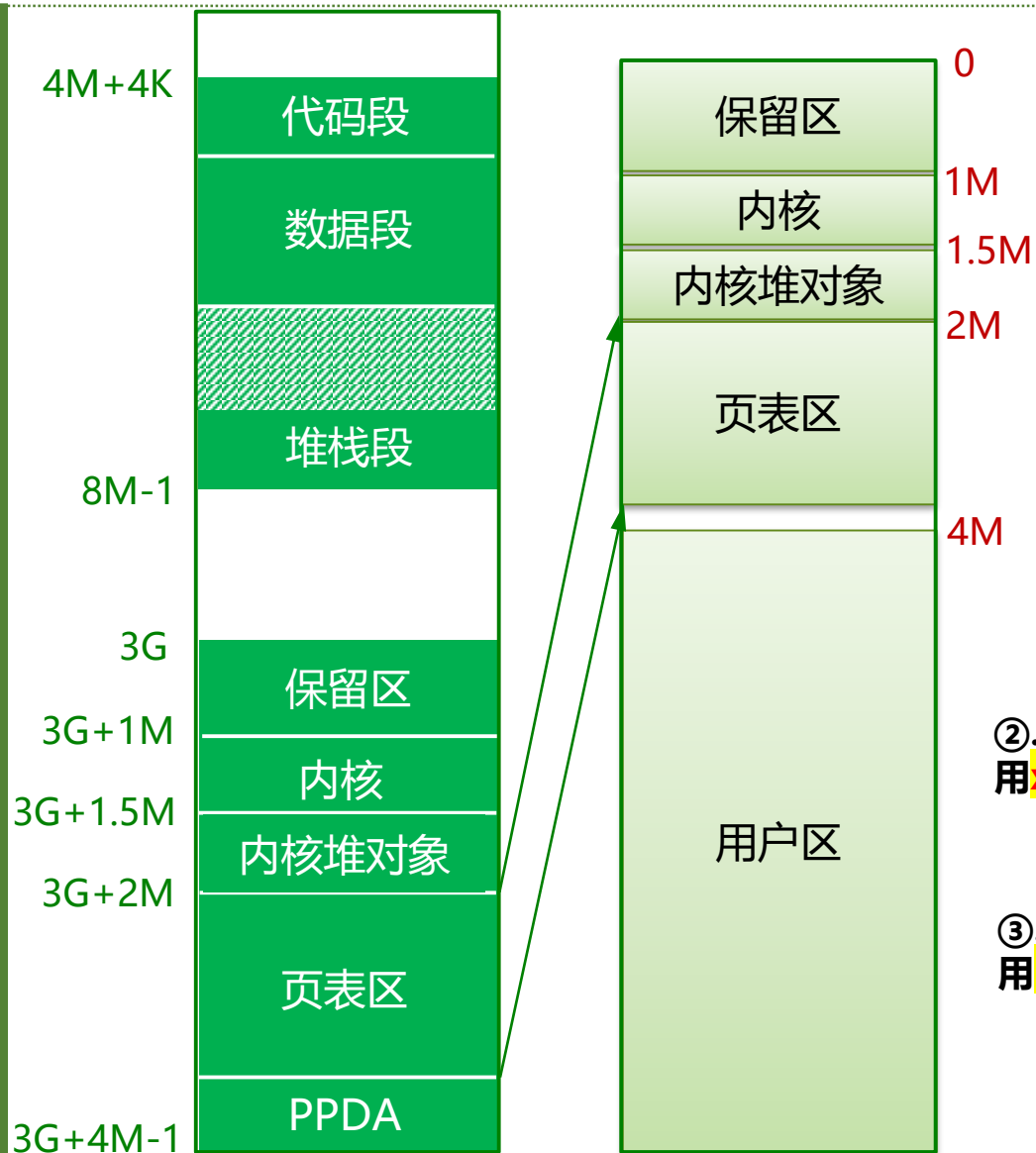
③. if $r/w == 1$, 直接用 p_addr 计算并写入



页表系统



页表的构成



虚实地址空间的分布沿用UNIX V6++的设计

修改要求：

- (1) 不使用交换区；
- (2) 直接使用相对表作为进程的物理页表

相对虚实地址映射表

	Page Base Addr	s/u	r/w	p
0#	xxx	x	x	x
...	...	...	...	...
1024#	xxx	x	x	x
1025#	0	1	0	1
1026#	1	1	0	1
1027#	1	1	1	1
	2	1	1	1
	全0			
2023#	3	1	1	1

②. if $r/w == 0$, 直接用 x_caddr 计算并写入

③. if $r/w == 1$, 直接用 p_addr 计算并写入

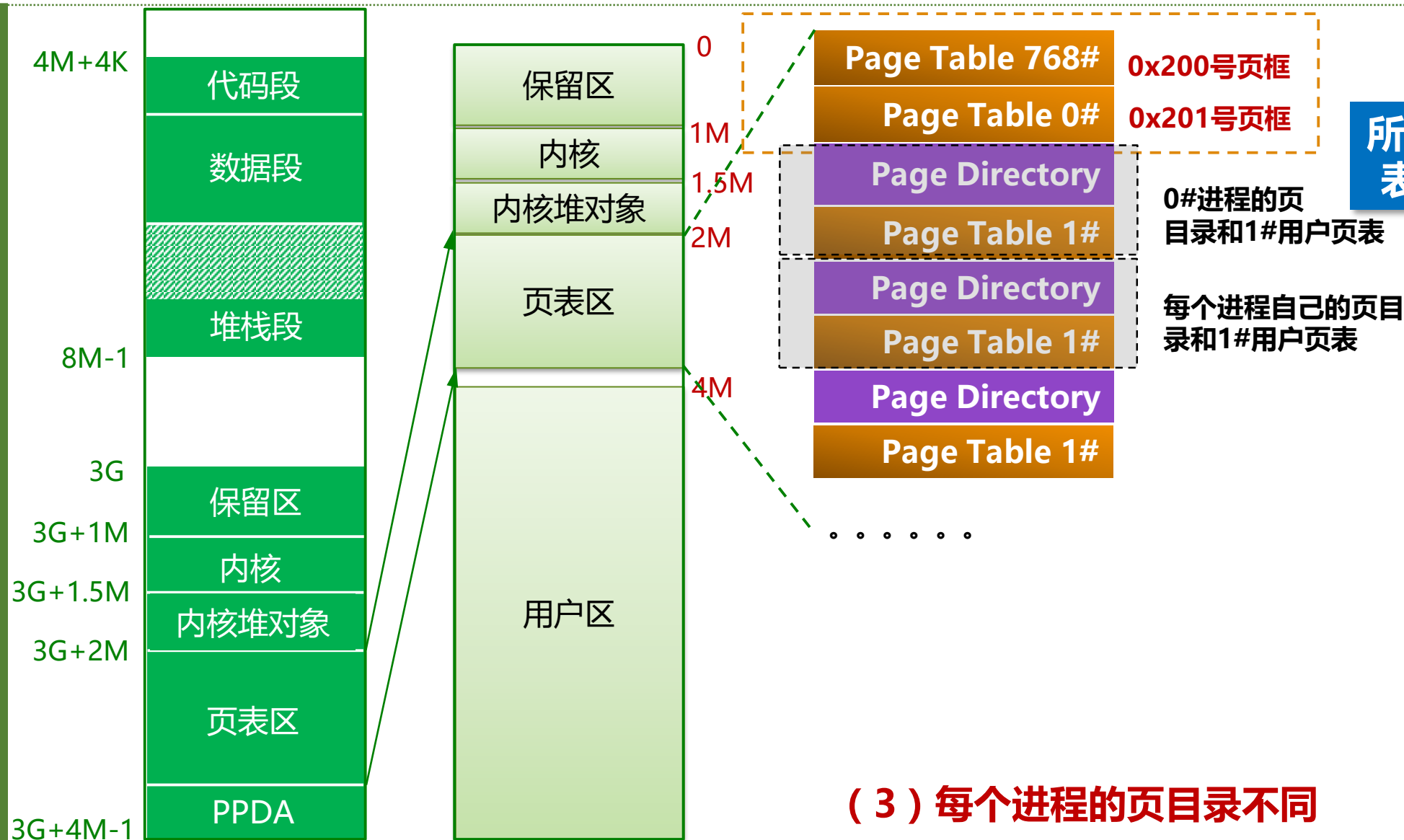
(3) 每个进程的页目录不同



页表系统



页表的构成



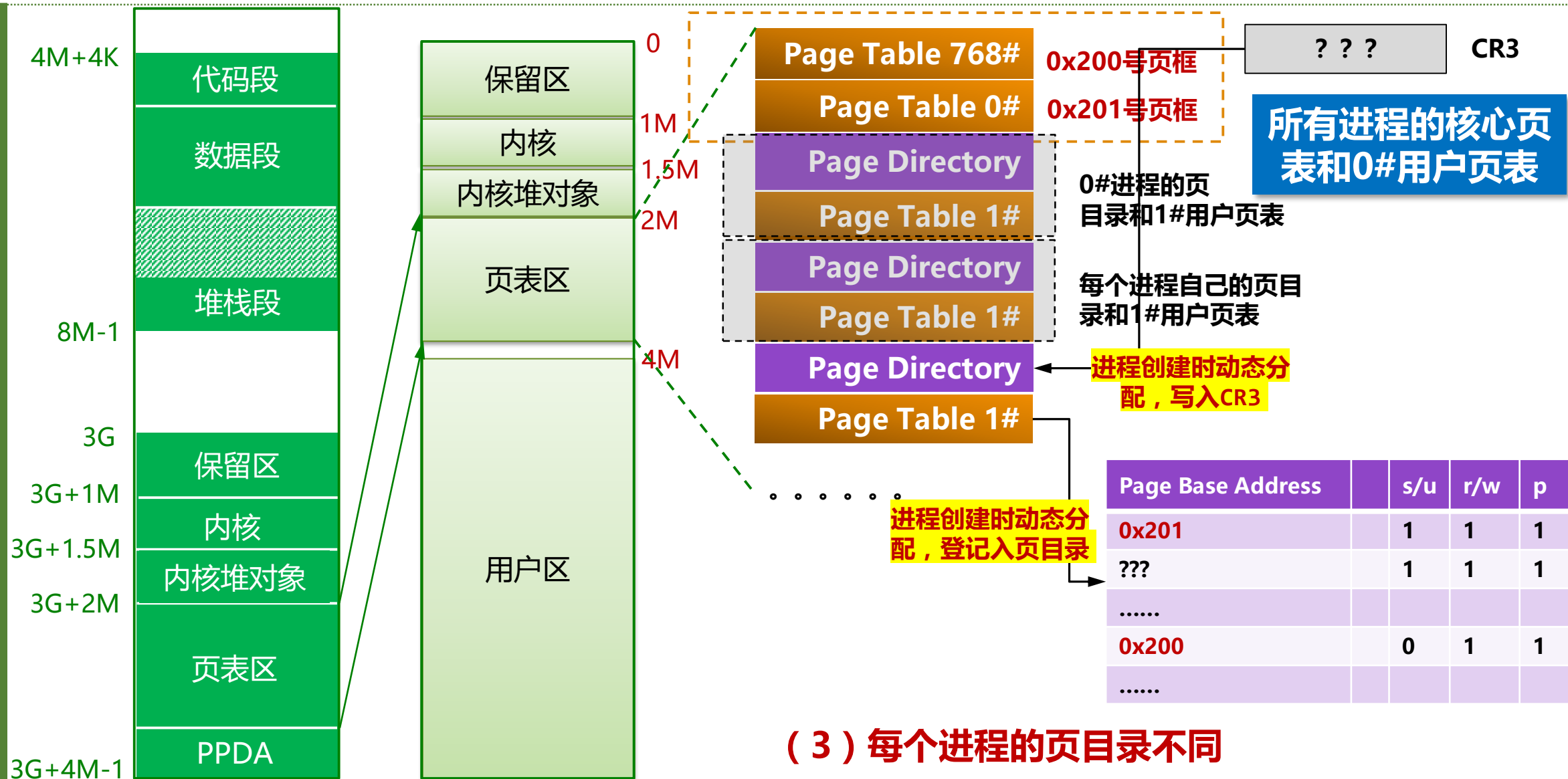
(3) 每个进程的页目录不同



页表系统



页表的构成

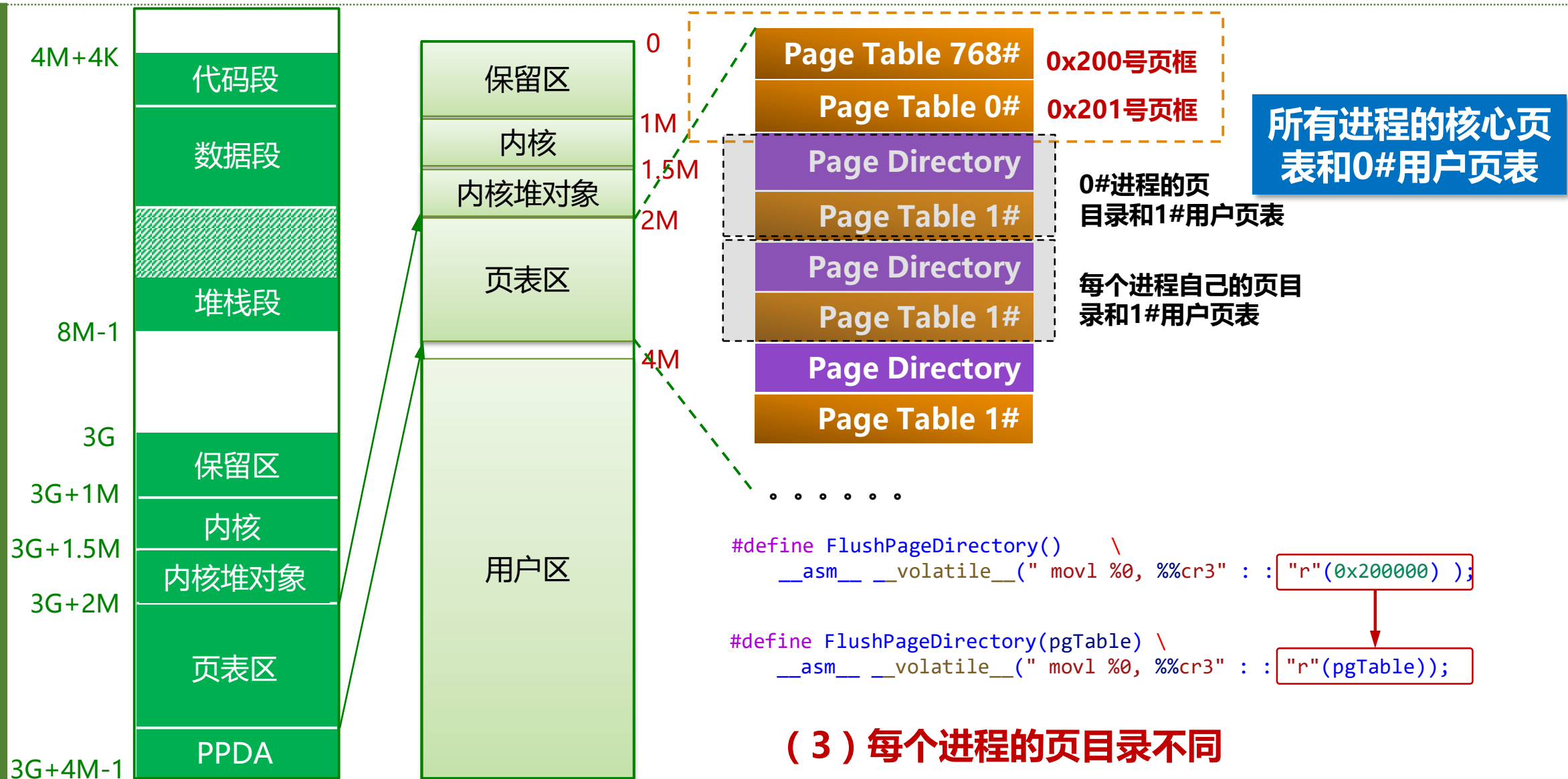




页表系统



页表的构成



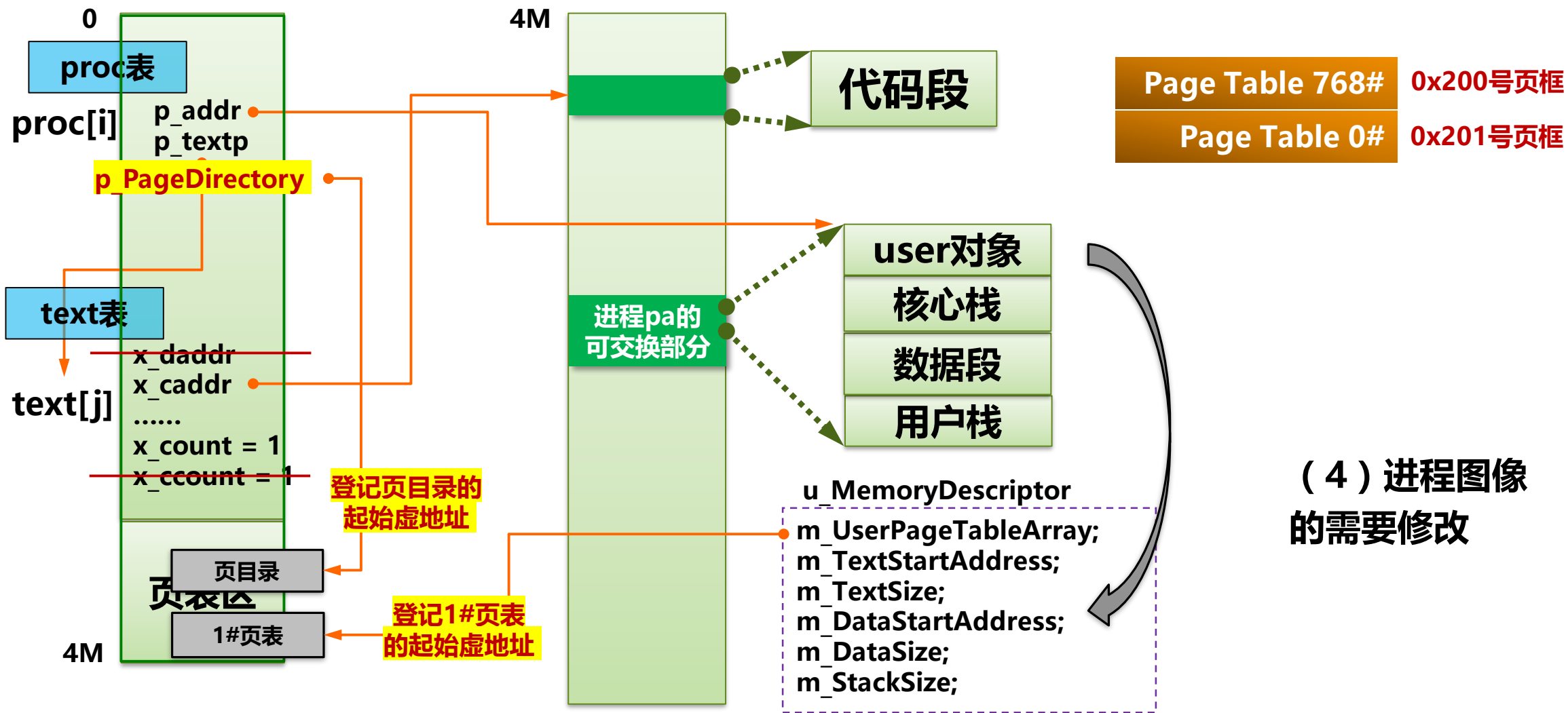
(3) 每个进程的页目录不同



页表系统



页表的构成





(5) 对Process类的修改

在Process类中增加：

```
public:
    PageDirectory *p_PageDirectory;          /* 指向每个进程页目录的起始逻辑地址 */
    unsigned long GetPageDirectoryPhyAddr();
```

```
unsigned long Process::GetPageDirectoryPhyAddr()
{
    return (unsigned long) this->p_PageDirectory - Machine::KERNEL_SPACE_START_ADDRESS;
}
```

指向进程页目录的逻辑地址

- 3G



(6) 对MemoryDescriptor的修改

```
class MemoryDescriptor
{
    /* 申请并初始化PageDirectory, 在做Map操作前使用 */
    + void Initialize();
    /* 在释放进程时, 需要调用该操作释放被占用的页表 */
    + void Release();
    + void ClearUserPageTable();

    + PageTable* GetUserPageTableArray();
    + unsigned long GetTextStartAddress();
    + unsigned long GetTextSize();
    + unsigned long GetDataStartAddress();
    + unsigned long GetDataSize();
    + unsigned long GetStackSize();

    + void MapTextEntrys(...);
    + void MapDataEntrys(...);
    + void MapStackEntrys(...);
    + unsigned int MapEntry(...);
    + bool EstablishUserPageTable(...);

    //修改MapToPageTable()函数将1#用户页表页框号写到私有的页目录
    + void MapToPageTable();
}
```

需要修改的接口函数：

因为m_UserPageTableArray只是指向一个页的用户页表，所以初始化时只分配一个页，回收时只回收一个页，清除时也只需要清除一个页

这些与相对表设置物理页表相关的接口函数都不再需要

需要新增的接口函数：

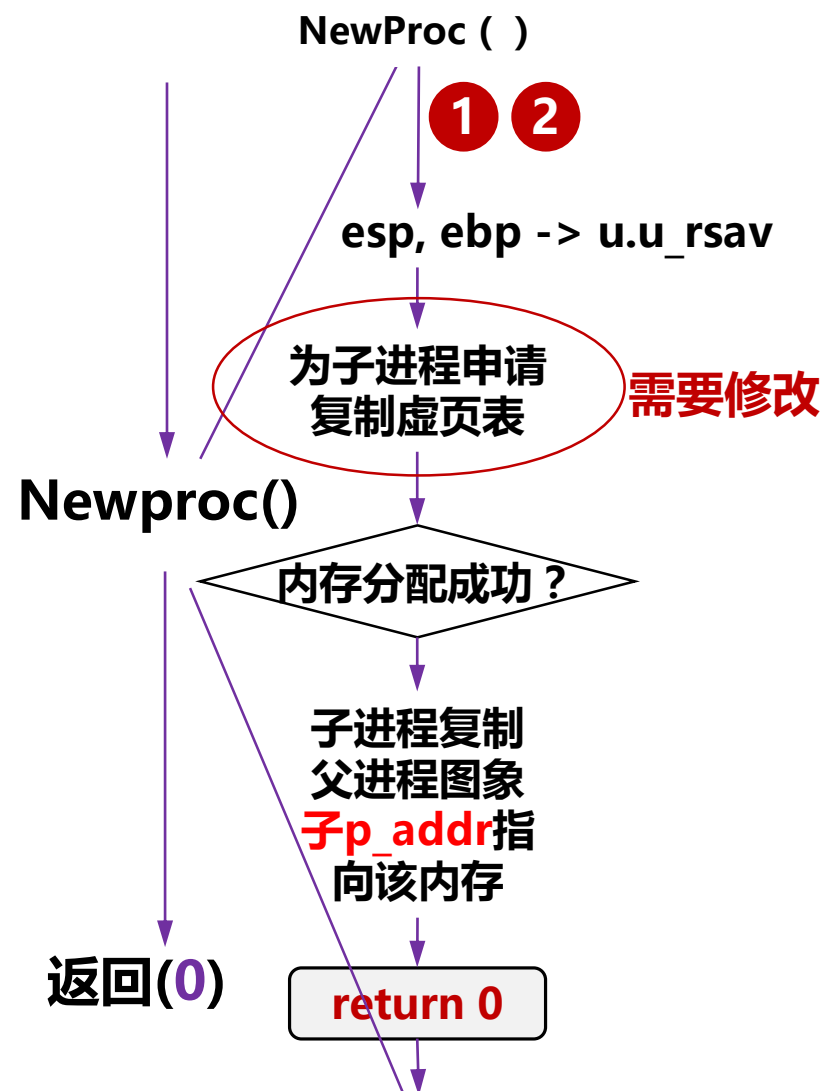
调用FlushPageDirectory函数将页目录的物理页框号写入CR3寄存器



(7) 修改进程生命周期中所有与分页机制相关的操作

创建进程时：

不需要考虑内存不够的情况！！！！





(7) 修改进程生命周期中所有与分页机制相关的操作

创建进程时：

```
PageDirectory *AllocPageDirectory();
```

p_PageDirectory
为进程分配一张页目录

Page Base Address		s/u	r/w	p
.....				
.....				

```
void InitProcPageDirectory(Process *proc, PageTable *privateUsrPageTable);
```

1#页表的逻辑地址

转成物理地址

Page Base Address		s/u	r/w	p
0x201		1	1	1
???		1	1	1
.....				
0x200		0	1	1
.....				



(7) 修改进程生命周期中所有与分页机制相关的操作

进程切换时：

```
#define SwtchUStruct(p) \  
    Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \  
        = (p)->p_addr / PageManager::PAGE_SIZE; \  
    FlushPageDirectory();
```



```
#define SwtchUStruct(p) \  
    Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress = \  
    (p)->p_addr >> 12; \  
    FlushPageDirectory((p)->GetPageDirectoryPhyAddr());
```



(7) 修改进程生命周期中所有与分页机制相关的操作

```
void ProcessManager::SetupProcessZero()
```

```
{
```

需要在最后添加：将0#进程的页目录设置在4M开始的第一个物理页框

```
}
```



将内存分配方式改为位示图



存储空间管理



	起始地址/KB	大小/KB
1	20	10
2	32	15
3	50	20

`public: MapNode map[512];` 最多包含512个表项的空闲分区表

`struct MapNode`

```
{  
    unsigned long m_Size;           其中每一个表项包含空闲分  
    unsigned long m_AddressIdx; 区的大小和分区的起始地址  
};
```



存储空间管理



用分区表管理内存

内核对象区、页表区、用户区均采用**可变分区方式**进行分配，
分别由**三张空闲分区表**管理：

`public:MapNode map[512];` 最多包含512个表项的空闲分区表

`struct MapNode`

```
{  
    unsigned long m_Size;           其中每一个表项包含空闲分  
    unsigned long m_AddressIdx; 区的大小和分区的起始地址  
};
```

内核堆对象区，其map初始化为：

`map[0]. m_AddressIdx = 1.5M , map[0]. m_Size = 0.5M。`

负责管理页表区，其map初始化为：

`map[0]. m_AddressIdx = 2M+16K`，即：从204号页框开始。

`map[0]. m_Size = 2M-16K`，即：剩余的页表区。

用户区，其map初始化为：

`map[0]. m_AddressIdx = 4M`，大小至整个物理内存。

大小	起始地址
0.5M	1.5M

大小	起始地址
2M-16K	2M+16K

大小	起始地址
至整个物理内存	4M





存储空间管理



```
struct MapNode
+ unsigned long m_Size;
+ unsigned long m_AddressIdx;
```

<MapNode.h>

1
2
3

	起始地址/KB	大小/KB
1	20	10
2	32	15
3	50	20

<Allocator.h>

<Allocator.cpp>

```
class Allocator
- static Allocator m_Instance;
+ unsigned long Alloc(MapNode map[], unsigned long size);
+ unsigned long Free(MapNode map[], unsigned long size, unsigned long addrIdx);
+ static Allocator& GetInstance();
```

用
位
示
图
管
理
内
存



存储空间管理



```
struct MapNode
+ unsigned long m_Size;
+ unsigned long m_AddressIdx;
```

<MapNode.h>

```
class BitMap
+ unsigned long m_AddressIdx;
+ int rows;
+ unsigned long long int map[256];

+ void set(unsigned long startAddr, unsigned long page_num)
```

	起始地址/KB	大小/KB
1	20	10
2	32	15
3	50	20

<Allocator.h>

<Allocator.cpp>

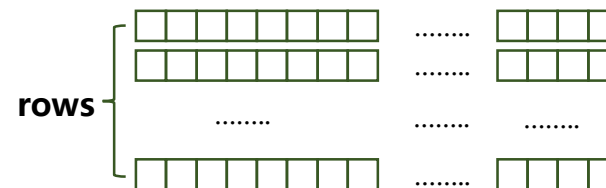
```
class Allocator
- static Allocator m_Instance;

+ unsigned long Alloc(MapNode map[], unsigned long size);
+ unsigned long Free(MapNode map[], unsigned long size, unsigned long addrIdx);
+ static Allocator& GetInstance();
```

```
class BitMapAllocator
- static BitMapAllocator m_Instance_bitmap;

+ unsigned long Alloc(BitMap &bitmap, unsigned long size);
+ unsigned long Free(BitMap &bitmap, unsigned long size, unsigned long addrIdx);
+ int is_free(BitMap &bitmap, int index);
+ void set_bit(BitMap &bitmap, int index, int value);
+ static BitMapAllocator &GetInstance();
```

64位无符号长整型



每个bit对应一个物理页框，4096字节；0表示空闲，1表示已分配



存储空间管理



用位示图管理内存

```
struct MapNode
{
+ unsigned long m_Size;
+ unsigned long m_AddressIdx;
}
```

<MapNode.h>

```
class BitMap
{
+ unsigned long m_AddressIdx;
+ int rows;
+ unsigned long long int map[256];
}
```

<PageManager.h>

<PageManager.cpp>

<Allocator.h>
<Allocator.cpp>

```
class Allocator
{
- static Allocator m_Instance;

+ unsigned long Alloc(...);
+ unsigned long Free(...);
+ static Allocator& GetInstance();
}
```

```
class BitMapAllocator
{
- static BitMapAllocator m_Instance_bitmap;

+ unsigned long Alloc(...);
+ unsigned long Free(...);
+ int is_free(BitMap &bitmap, int index);
+ void set_bit(BitMap &bitmap, int index, int value);
+ static BitMapAllocator &GetInstance();
}
```

class PageManager

```
+ MapNode map[PageManager::MEMORY_MAP_ARRAY_SIZE];
- Allocator* m_pAllocator;
```

```
/* 设置m_pAllocator */
+ PageManager(Allocator* allocator);
```

```
/* 初始化分区表map */
+ int Initialize();
```

```
/* 调用m_pAllocator->Alloc完成内存分配 */
+ unsigned long AllocMemory(unsigned long size);
```

```
/* 调用m_pAllocator->Free完成内存回收 */
+ unsigned long FreeMemory(unsigned long size,
                           unsigned long memoryStartAddress);
```



存储空间管理



用位示图管理内存

```
struct MapNode
{
+ unsigned long m_Size;
+ unsigned long m_AddressIdx;
}
```

<MapNode.h>

```
class BitMap
{
+ unsigned long m_AddressIdx;
+ int rows;
+ unsigned long long int map[256];
}
```

class BitMap

<Allocator.h>

<Allocator.cpp>

class Allocator

- static Allocator m_Instance;

```
+ unsigned long Alloc(...);
+ unsigned long Free(...);
+ static Allocator& GetInstance();
```

class BitMapAllocator

- static BitMapAllocator m_Instance_bitmap;

```
+ unsigned long Alloc(...);
+ unsigned long Free(...);
+ int is_free(BitMap &bitmap, int index);
+ void set_bit(BitMap &bitmap, int index, int value);
+ static BitMapAllocator &GetInstance();
```

<PageManager.h>

<PageManager.cpp>

class PageManager

```
+ MapNode map[PageManager::MEMORY_MAP_ARRAY_SIZE];
- BitMap Allocator* m_pAllocator;
+ BitMap bitmap;
- BitMap Allocator* bm_pAllocator;
```

```
/* 设置m_pAllocator */
+ PageManager(Allocator* allocator);
```

```
/* 设置bm_pAllocator */
+ PageManager(BitMapAllocator *allocator);
```

```
/* 初始化位示图bitmap */
+ int Initialize();
```

```
/* 调用m_pAllocator->Alloc完成内存分配 */
+ unsigned long AllocMemory(...);
```

```
/* 调用m_pAllocator->Free完成内存回收 */
+ unsigned long FreeMemory(...);
```

需要添加关于位示图的数据成员

添加一个使用位示图的构造函数

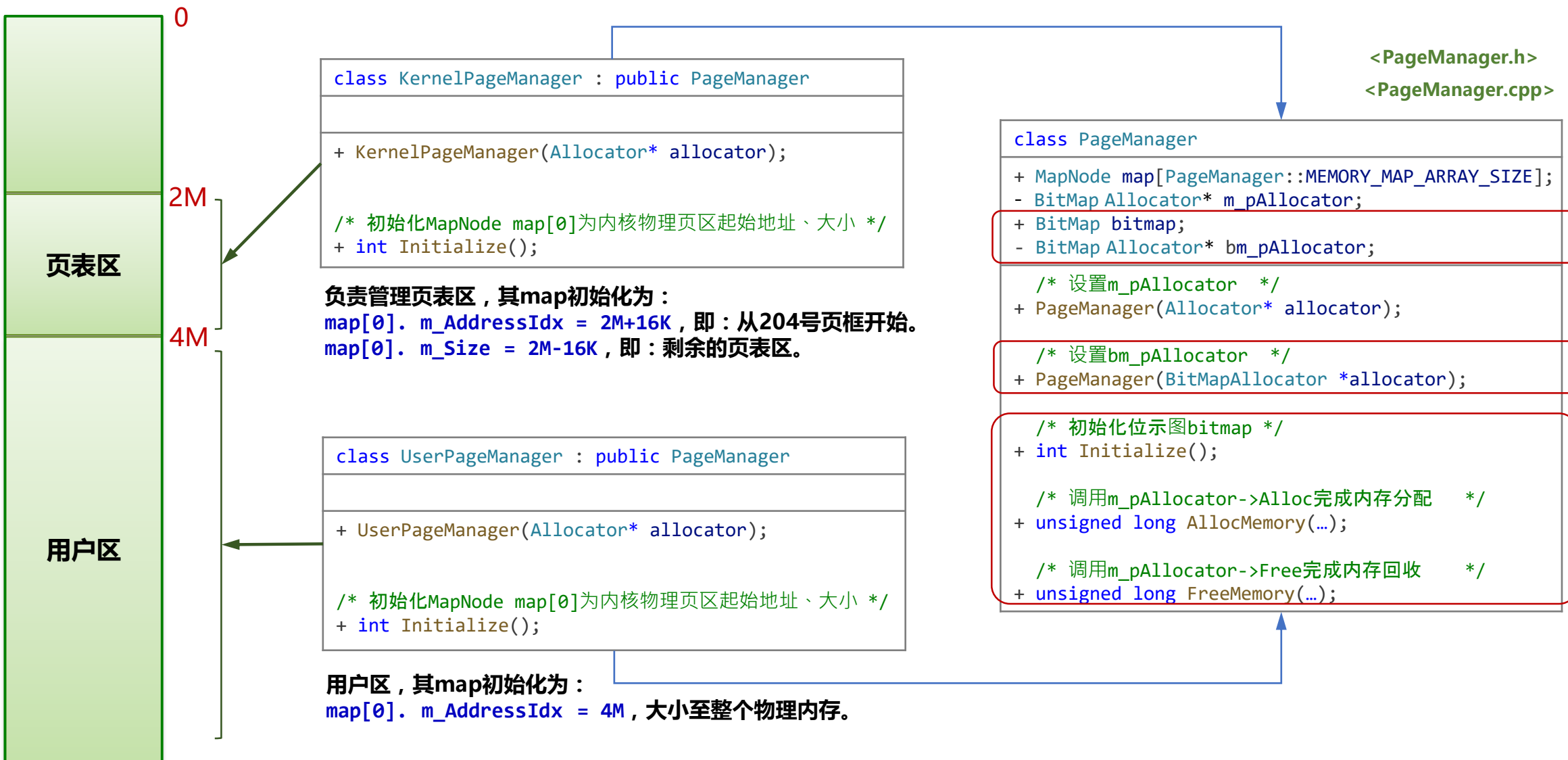
接口不变，所有实现需修改为针对位示图的实现



存储空间管理



用位示图管理内存

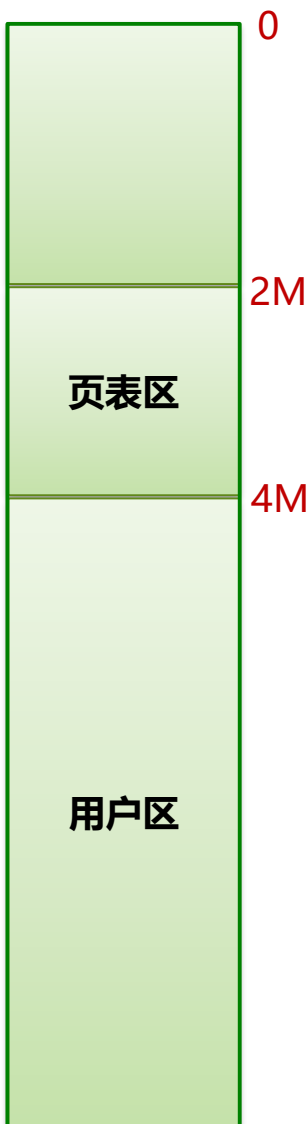




存储空间管理



用位示图管理内存



```
class KernelPageManager : public PageManager
```

```
+ KernelPageManager(Allocator* allocator);
```

```
+ KernelPageManager(BitMapAllocator *allocator);  
/* 初始化MapNode map[0]为内核物理页区起始地址、大小 */  
+ int Initialize();
```

添加构造函数，修改初始化操作

负责管理页表区，其bitmap初始化为：

map[0]. m_AddressIdx = 2M+16K，即：从204号页框开始。
map[0]. m_Size = 2M-16K，即：剩余的页表区。

```
class UserPageManager : public PageManager
```

```
+ UserPageManager(Allocator* allocator);
```

```
+ UserPageManager(BitMapAllocator *allocator);  
/* 初始化MapNode map[0]为内核物理页区起始地址、大小 */  
+ int Initialize();
```

添加构造函数，修改初始化操作

用户区，其bitmap初始化为：

map[0]. m_AddressIdx = 4M，大小至整个物理内存。

<PageManager.h>

<PageManager.cpp>

```
class PageManager
```

```
+ MapNode map[PageManager::MEMORY_MAP_ARRAY_SIZE];  
- BitMap Allocator* m_pAllocator;  
+ BitMap bitmap;  
- BitMap Allocator* bm_pAllocator;
```

```
/* 设置m_pAllocator */  
+ PageManager(Allocator* allocator);
```

```
/* 设置bm_pAllocator */  
+ PageManager(BitMapAllocator *allocator);
```

```
/* 初始化位示图bitmap */  
+ int Initialize();
```

```
/* 调用m_pAllocator->Alloc完成内存分配 */  
+ unsigned long AllocMemory(...);
```

```
/* 调用m_pAllocator->Free完成内存回收 */  
+ unsigned long FreeMemory(...);
```



离散化存储